

# Porting cilium-agent to Windows

Stages followed to build & run the agent on Windows nodes with eBPF-for-Windows (efw)

---

## Stage 0 — Baseline & strategy

*Goal: build the agent under GOOS=windows and run it on Windows nodes where efw provides the BPF maps, without changing Linux behaviour.*

- Split OS-specific code into `*_linux.go` / `*_windows.go` (and `*_other.go`) using Go build tags.
- Keep Linux code paths byte-for-byte unchanged; Windows adds parallel implementations only.

## Stage 1 — Replace Linux APIs with generic cross-platform equivalents

*Goal: remove hard Linux dependencies that have portable alternatives.*

- Replaced `golang.org/x/sys/unix` usage with generic Go stdlib APIs that work on Windows and Linux.
- Swapped Linux-specific constants for portable / Windows-safe constants.
- Where no generic API existed, introduced build-tagged wrappers.

## Stage 2 — Make pkg/bpf work on Windows (minimum diff vs. winebpfmap)

*Goal: reuse the working Windows BPF calls from winebpfmap with the smallest change to pkg/bpf.*

- Compared winebpfmap (working Windows BPF) against the Linux pkg/bpf implementation.
- Adopted **upstream ebpf-go v0.22.0** (native efw support — no fork, no cgo), then added the minimum:
  - – Pin path → `/ebpf/global/<name>` (forward slashes) so the agent shares the datapath's objects.
  - – **Prefer-legacy map-name alias** so the agent reuses the datapath's v1 maps.
  - – Map-type/flag translation (Linux `ebpf.Hash/...` → `ebpf.Windows*`, drop `BPF_F_*` flags).
  - – Non-batch iteration fallback (efw has no `BPF_MAP_LOOKUP_BATCH`).

## Stage 3 — Use Windows system APIs as alternates

*Source: wcnagent\_windows\_apis.xlsx*

- For APIs with a Windows equivalent: Linux impl in `*_linux.go` (linux tag), Windows impl in `*_windows.go` (windows tag).
- Ignored the **Cnc\*** entries — direct BPF calls are already in place.

## Stage 4 — Implement Linux APIs with Windows APIs

*Source: wcnagent\_porting\_design.xlsx*

- For the subset that could be re-implemented, added real Windows implementations in `*_windows.go`.
- Kept the corresponding Linux implementations in `*_linux.go`.

## Stage 5 — No-op stubs for unportable pieces

*Goal: satisfy the build where a Windows implementation is not needed.*

- Added empty / no-op `*_windows.go` implementations; Linux versions stay in `*_linux.go`.
- Silenced Linux-only runtime errors: iptables, CT/NAT GC batch API, health, utime.

## Post-port runtime bring-up & verification

- Fixed Services not programming the efw LB maps (pin-path + map-version-skew) — **verified working**.
  - Fixed LoadBalancer/external VIPs not programmed: `isDatapathCandidate` → `platformDatapathCandidate` (LoadBalancer programmed when kube-proxy-replacement is off) — **verified working**.
  - ClusterIP + LoadBalancer VIP reachable; curl succeeds against demo service.
  - Whole repo **go build ./...** and **go vet ./...** green on both Windows and Linux.
-

| Stage | Focus                                   | Build-tag split                           |
|-------|-----------------------------------------|-------------------------------------------|
| 1     | Generic cross-platform APIs / constants | wrappers where needed                     |
| 2     | pkg/bpf on efw via ebpf-go              | bpffs / maptype / mapbatch / legacy alias |
| 3     | Windows API alternates                  | *_linux.go / *_windows.go                 |
| 4     | Linux APIs re-implemented on Windows    | *_linux.go / *_windows.go                 |
| 5     | No-op stubs to build                    | *_linux.go / *_windows.go                 |